

Módulo 5: Lenguaje SQL

Como dijimos anteriormente SQL es el idioma estándar para comunicarse con bases de datos relacionales, permitiendo tanto la gestión de la estructura como la manipulación de los datos que contienen. Un programador necesita conocer SQL porque casi todas las aplicaciones modernas interactúan con bases de datos para almacenar, gestionar y recuperar información. SQL es el lenguaje estándar para comunicarse con estas bases de datos relacionales. En esta última semana veremos consultas combinando diferentes tablas usando “JOIN”, “UNION” y operaciones de conjuntos sobre múltiples tablas

Semana 3

PARTE 1:

Combinando datos

En una base de datos relacional, la información se distribuye en múltiples tablas para evitar la redundancia y optimizar el almacenamiento. Los JOINS son un mecanismo esencial de SQL que permite combinar datos de dos o más tablas en una única consulta, basándose en una relación lógica entre ellas, generalmente una clave foránea que referencia a una clave primaria. Este proceso nos permite obtener información completa que reside en tablas separadas.

La sentencia UNION por su parte permite ofrecer como resultado un conjunto de columnas que puede tener orígenes en tablas distintas o, respetando algunos criterios, en instrucciones SELECT diferentes.

La combinación de estas dos formas de consulta es también posible, lo que ofrece una gran posibilidad de combinaciones para obtener resultados.

Sentencia JOIN

Comencemos con la sentencia que permite juntar en una consulta datos de diferentes tablas. En general el proceso de normalización busca evitar la redundancia de datos y la creación de tablas de manera eficiente. Esta sentencia permite recuperar los datos distribuidos en diferentes tablas, utilizando para ello alguna columna o más columnas comunes entre tablas.

La sintaxis general es la siguiente:

```
SELECT <columnas_a_seleccionar>  
FROM <tabla_izquierda>  
JOIN <tabla_derecha>  
ON <condicion_de_union>;
```

Donde en FROM se coloca la tabla de origen, en JOIN se indica la tabla relacionada y dentro de ON se especifican las columnas que deben coincidir. El resultado es la combinación de columnas seleccionadas de las tablas indicadas en nuevas filas, que pueden tener datos redundantes.

Supongamos que tenemos una tabla con autores de libros que tiene como clave principal una columna AutorID y otra tabla de Libros que tiene la columna AutorID para relacionarlos.

La consulta:

```
SELECT A.Nombre, L.Titulo  
FROM Autores AS A  
INNER JOIN Libros AS L  
ON A.AutorID = L.AutorID;
```

Devuelve las columnas nombre de autor y título del libro, asociando cada autor con sus respectivos libros. Aquí se combinan las columnas de dos tablas diferentes y la columna que las relaciona es AutorID, que está presente en ambas tablas. Si existe algún autor que no tenga libros en la tabla, no aparece en esta consulta.

Pero JOIN tiene algunas variantes de comportamientos según se los datos que se necesite recuperar. Puedes querer mantener los datos de la tabla origen aun cuando no tengas coincidencias en la tabla relacionado o viceversa. Entonces veamos una comparación entre ellos

Tipo de JOIN	Lógica de la Unión	¿Qué Registros se Devuelven?	¿Cuándo se Presentan Valores NULL?
INNER JOIN	Intersección de conjuntos	Solo los registros con valores coincidentes en ambas tablas	Nunca.
LEFT JOIN	Todos los de la izquierda + Coincidentes	Todos los registros de la tabla izquierda y los que coinciden en la derecha.	En los campos de la tabla derecha cuando no hay coincidencia.
RIGHT JOIN	Todos los de la derecha + Coincidentes	Todos los registros de la tabla derecha y los que coinciden en la izquierda.	En los campos de la tabla izquierda cuando no hay coincidencia.

Veamos algunos ejemplos:

LEFT JOIN Manteniendo la Tabla "Izquierda"

El LEFT JOIN (también conocido como LEFT OUTER JOIN) devuelve todos los registros de la tabla izquierda, junto con los registros coincidentes de la tabla derecha. Si no hay una coincidencia en la tabla de la derecha para un registro de la izquierda, los campos de la tabla derecha se rellenan con valores NULL

Sintaxis:

```
SELECT <columnas_a_seleccionar>  
FROM <tabla_izquierda>  
LEFT JOIN <tabla_derecha>  
ON <condicion_de_union>;
```

Ejemplo Listar todos los autores y los títulos de sus libros publicados, si tienen.

```
SELECT A.Nombre, L.Titulo  
FROM Autores AS A  
LEFT JOIN Libros AS L  
ON A.AutorID = L.AutorID;
```

Resultado: Esta consulta devolverá todos los autores de la tabla Autores. Para los autores que tienen libros, se mostrará el título del libro correspondiente. Para los que no, el campo Título mostrará NULL.

RIGHT JOIN: Manteniendo la Tabla "Derecha"

El RIGHT JOIN (también conocido como RIGHT OUTER JOIN) es la contraparte simétrica del LEFT JOIN. Devuelve todos los registros de la tabla derecha, junto con los registros coincidentes de la tabla izquierda. Si no hay coincidencia en la tabla de la izquierda, los campos de la tabla izquierda se rellenan con valores NULL.

Sintaxis:

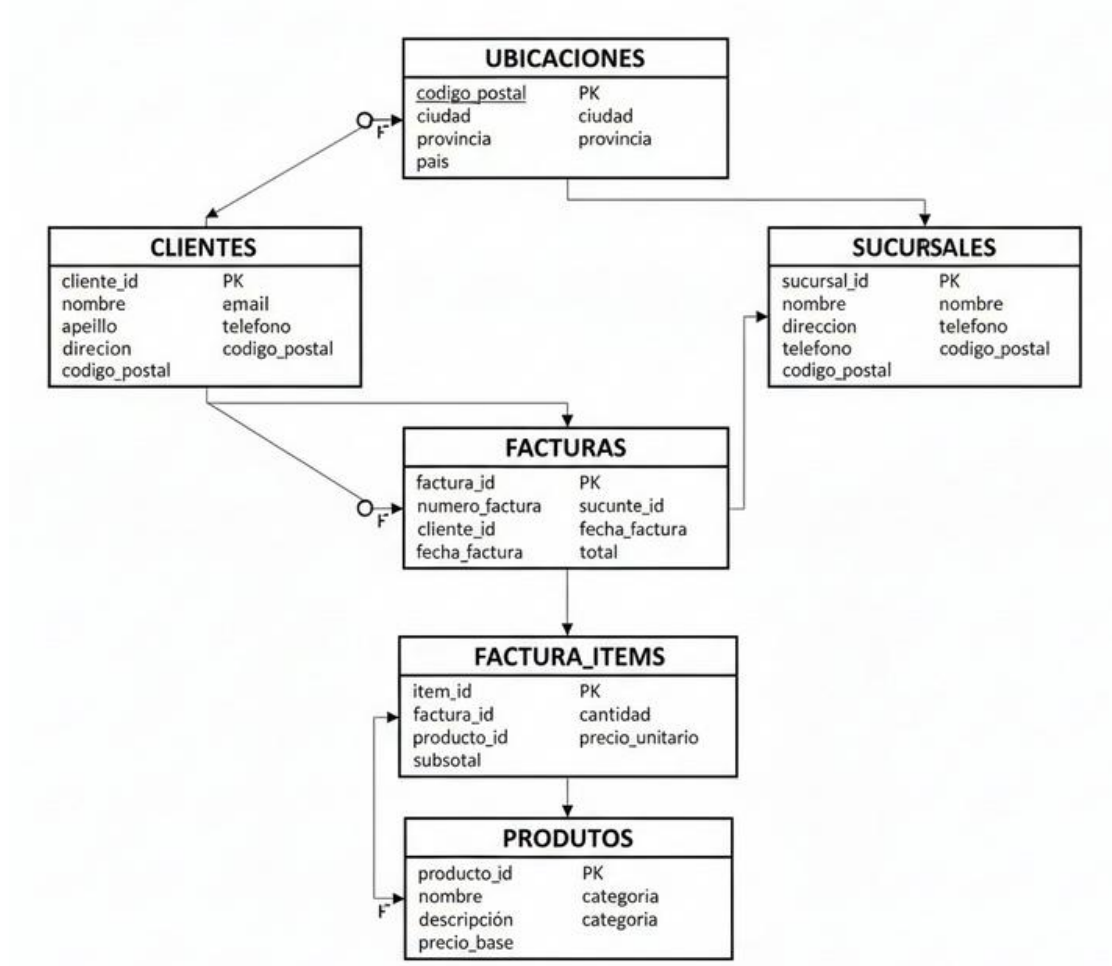
```
SELECT <columnas_a_seleccionar>  
FROM <tabla_izquierda>  
RIGHT JOIN <tabla_derecha>  
ON <condicion_de_union>;
```

Ejemplo Listar todos los libros y el nombre de su autor correspondiente.

```
SELECT A.Nombre, L.Titulo  
FROM Autores AS A  
RIGHT JOIN Libros AS L  
ON A.AutorID = L.AutorID;
```

Obtener información de facturación

Supongamos que tenemos las tablas de siguiente diagrama



Para encontrar ciudades donde se vende un producto

```

SELECT DISTINCT
  u.ciudad,
  u.provincia,
  u.pais,
  COUNT(fi.item_id) AS total_ventas,
  SUM(fi.cantidad) AS cantidad_total_vendida
FROM productos p
INNER JOIN factura_items fi ON p.producto_id = fi.producto_id
INNER JOIN facturas f ON fi.factura_id = f.factura_id
INNER JOIN clientes c ON f.cliente_id = c.cliente_id
INNER JOIN ubicaciones u ON c.codigo_postal = u.codigo_postal
WHERE p.nombre = 'Laptop Gaming' -- Producto específico
GROUP BY u.codigo_postal, u.ciudad, u.provincia, u.pais
ORDER BY cantidad_total_vendida DESC;
  
```

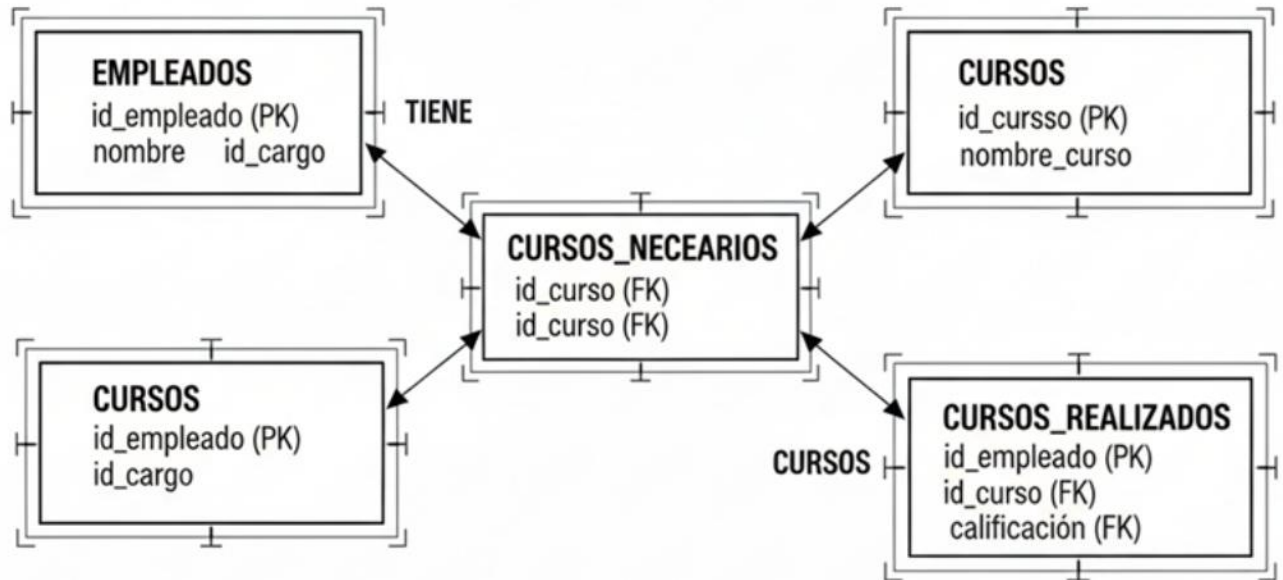
Incluimos los datos de las sucursales a la consulta y un rango de fechas

```
SELECT
    u.ciudad,
    u.provincia,
    COUNT(DISTINCT f.cliente_id) AS clientes_diferentes,
    SUM(fi.cantidad) AS unidades_vendidas,
    SUM(fi.subtotal) AS ingresos_totales
FROM productos p
INNER JOIN factura_items fi ON p.producto_id = fi.producto_id
INNER JOIN facturas f ON fi.factura_id = f.factura_id
INNER JOIN clientes c ON f.cliente_id = c.cliente_id
INNER JOIN ubicaciones u ON c.codigo_postal = u.codigo_postal
WHERE p.nombre LIKE '%Laptop%' -- Productos que contengan "Laptop"
AND f.fecha_factura BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY u.codigo_postal, u.ciudad, u.provincia
HAVING unidades_vendidas > 5 -- Solo ciudades con más de 5 unidades vendidas
ORDER BY ingresos_totales DESC;
```

Para listar el Top 10 de ciudades de un producto

```
SELECT
    p.nombre AS producto,
    u.ciudad,
    u.provincia,
    u.pais,
    SUM(fi.cantidad) AS total_vendido,
    COUNT(DISTINCT f.factura_id) AS facturas_diferentes,
    COUNT(DISTINCT f.cliente_id) AS clientes_diferentes,
    ROUND(SUM(fi.subtotal), 2) AS ingresos_total
FROM productos p
INNER JOIN factura_items fi ON p.producto_id = fi.producto_id
INNER JOIN facturas f ON fi.factura_id = f.factura_id
INNER JOIN clientes c ON f.cliente_id = c.cliente_id
INNER JOIN ubicaciones u ON c.codigo_postal = u.codigo_postal
WHERE p.producto_id = 1
GROUP BY p.producto_id, p.nombre, u.codigo_postal, u.ciudad, u.provincia, u.pais
ORDER BY total_vendido DESC
LIMIT 10;
```

Veamos otro ejemplo, en este caso obtener lo que falta, consideremos que tenemos las siguientes tablas, donde cursos necesarios relaciona los cargos con los distintos cursos que correspondan al mismo.



Necesitamos encontrar que capacitaciones le faltan aprobar con nota superior a 8 un empleado para satisfacer los requisitos del cargo que ocupa.

Para ello tenemos que determinar todos los cursos que cada empleado DEBERÍA tener (según su cargo) y comparar esa lista con los cursos que REALMENTE tiene aprobados (calificación ≥ 8).

Comencemos analizando las partes de la consulta, primero obteniendo los cursos que debería tener un empleado según el cargo que tiene.

```
SELECT
  e.id_empleado,
  e.nombre AS nombre_empleado,
  c.id_cargo,
  cn.id_curso
FROM empleados e
INNER JOIN cursos_necesarios cn ON e.id_cargo = cn.id_cargo;
```

Esta consulta nos da una lista de todos los (empleado, curso) que son obligatorios.

Ahora, obtenemos los cursos que cada empleado ya aprobó (calificación ≥ 8).

```
SELECT
  cr.id_empleado,
  cr.id_curso
FROM cursos_realizados cr
WHERE cr.calificacion >= 8;
```

La Consulta Final (Encontrar los Faltantes)

La forma más clara de hacerlo es usando un LEFT JOIN entre la lista de cursos obligatorios y la de cursos aprobados. Donde no haya coincidencia (es decir, `curso_aprobado.id_curso IS NULL`), significará que el curso falta.

```
SELECT
    emp.id_empleado,
    emp.nombre AS nombre_empleado,
    car.nombre_cargo,
    cur.id_curso,
    cur.nombre_curso,
    cr.calificacion -- Mostramos la calificación si existe, aunque sea baja
FROM empleados emp -- Unimos para obtener el nombre del cargo
INNER JOIN cargos car ON emp.id_cargo = car.id_cargo -- Unimos para obtener todos los cursos obligatorios para el cargo
del empleado
INNER JOIN cursos_necesarios cn ON emp.id_cargo = cn.id_cargo -- Unimos para obtener el nombre del curso
INNER JOIN cursos cur ON cn.id_curso = cur.id_curso -- LEFT JOIN CRUCIAL: Traemos los cursos realizados que
coincidan (empleado + curso) y que estén aprobados. Si no existe, los valores serán NULL.
LEFT JOIN cursos_realizados cr ON emp.id_empleado = cr.id_empleado
    AND cn.id_curso = cr.id_curso
    AND cr.calificacion >= 8
-- FILTRO: Nos quedamos solo con los registros donde NO HAY un curso aprobado.
WHERE cr.id_curso IS NULL;
```

Parte 2

La sentencia UNION

Para entender UNION ALL, piensa en ella como una herramienta para combinar los resultados de dos o más consultas SELECT en un solo conjunto de resultados. A diferencia de UNION, que elimina las filas duplicadas, UNION ALL incluye todas las filas de cada consulta, incluso si son idénticas.

Cómo se usa UNION ALL

La sintaxis básica es bastante sencilla. Solo necesitas escribir una sentencia SELECT, seguida de la palabra clave UNION ALL, y luego otra sentencia SELECT. Puedes encadenar tantas consultas como necesites.

```
SELECT columna1, columna2, ...  
FROM tabla1  
WHERE condicion1  
UNION ALL  
SELECT columna1, columna2, ...  
FROM tabla2  
WHERE condicion2;
```

Imagina que tienes dos tablas: empleados_ventas y empleados_marketing. Quieres obtener una lista completa de todos los empleados de ambos departamentos.

```
SELECT nombre, apellido, 'Ventas' AS departamento  
FROM empleados_ventas  
UNION ALL  
SELECT nombre, apellido, 'Marketing' AS departamento  
FROM empleados_marketing;
```

En este ejemplo, la columna 'Ventas' (y 'Marketing') se agrega a la salida para identificar a qué departamento pertenece cada empleado.

Restricciones Importantes

Para que UNION ALL funcione correctamente, hay dos reglas cruciales que debes seguir:

Mismo número de columnas: Cada consulta SELECT que combines debe tener exactamente el mismo número de columnas.

Tipos de datos compatibles: Las columnas correspondientes en cada consulta deben tener tipos de datos compatibles entre sí. Por ejemplo, la primera columna de la primera consulta debe ser compatible con la primera columna de la segunda consulta, y así sucesivamente.

Si no cumples estas restricciones, el motor de base de datos devolverá un error.

¿Cuándo usar UNION ALL en lugar de UNION?

Usa UNION ALL cuando sabes que no hay (o no te importa si hay) filas duplicadas y necesitas un rendimiento más rápido. El proceso de eliminar duplicados de UNION requiere un procesamiento adicional que consume más recursos. Usa UNION solo cuando sea fundamental eliminar las filas duplicadas.

Usando GROUP BY

Puedes usar la cláusula GROUP BY en una sola consulta para agrupar filas idénticas y, de esta forma, obtener un conjunto de resultados sin duplicados. Es una forma efectiva de combinar la lógica de varias consultas en una sola.


```
SELECT columna1, columna2, ...  
FROM (  
    -- Tu primera consulta SELECT  
    SELECT columna1, columna2, ... FROM tabla1  
    UNION ALL  
    -- Tu segunda consulta SELECT  
    SELECT columna1, columna2, ... FROM tabla2  
) AS T  
GROUP BY columna1, columna2, ...;
```

En este caso, aunque se usa UNION ALL internamente para combinar los resultados, es el GROUP BY final el que se encarga de la eliminación de duplicados.

Usando DISTINCT

El operador DISTINCT se utiliza para eliminar filas duplicadas de una única consulta SELECT. Si bien no es una alternativa directa para combinar varias consultas como lo hace UNION, puedes usarlo en una subconsulta o una CTE (Common Table Expression) para filtrar duplicados antes de la unión o en la unión misma.

```
SELECT DISTINCT columna1, columna2, ...  
FROM tabla1  
UNION ALL  
SELECT DISTINCT columna1, columna2, ...  
FROM tabla2;
```

Este ejemplo no elimina los duplicados entre los dos conjuntos de resultados combinados, solo dentro de cada conjunto antes de la unión. Para eliminar los duplicados después de la unión, usarías UNION.

Sentencia WITH

Puedes definir múltiples subconsultas dentro de una sola sentencia WITH. Cada subconsulta se convierte en un CTE con su propio nombre, y puedes unirlos usando UNION o UNION ALL en la consulta principal.

Expresión Común de Tabla (CTE): Cada subconsulta precedida por WITH y seguida de su propio nombre se convierte en una Expresión Común de Tabla (CTE), que es un conjunto de resultados temporal y con nombre.

La sintaxis es simple:

```
WITH  
    CTE_Nombre1 AS (  
        -- Primera subconsulta aquí  
        SELECT columna1, columna2, ...  
        FROM tablaA  
    ),  
    CTE_Nombre2 AS (  
        -- Segunda subconsulta aquí  
        SELECT columna1, columna2, ...  
        FROM tablaB  
    )  
-- Consulta principal que combina los CTEs  
SELECT * FROM CTE_Nombre1  
UNION ALL
```

```
SELECT * FROM CTE_Nombre2;
```

Consideremos que necesitamos consolidar la información de ventas de productos vendidos en tiendas físicas y en línea para un informe.

```
WITH
  VentasFisicas AS (
    SELECT
      producto_id,
      SUM(cantidad) AS total_vendido
    FROM
      ventas_tienda_fisica
    GROUP BY
      producto_id
  ),
  VentasOnline AS (
    SELECT
      producto_id,
      SUM(cantidad) AS total_vendido
    FROM
      ventas_online
    GROUP BY
      producto_id
  )
SELECT
  producto_id,
  total_vendido
FROM
  VentasFisicas
UNION ALL
SELECT
  producto_id,
  total_vendido
FROM
  VentasOnline;
```

En esta consulta, VentasFisicas y VentasOnline son CTEs que agrupan las ventas por producto. La consulta principal luego usa UNION ALL para combinarlas en un único resultado.

Joins Versátiles con CTEs

Una de las mayores ventajas de las CTEs es que puedes usarlas como si fueran tablas reales en tus JOINS, lo que permite una lógica de consulta más avanzada y legible. Por ejemplo, puedes unir un CTE con una tabla principal o con otro CTE.

SQL

```
WITH
  empleados_departamento AS (
    SELECT
      e.nombre,
      e.apellido,
      d.nombre_departamento
    FROM
      empleados e
    INNER JOIN
```

```
    departamentos d ON e.id_departamento = d.id
  WHERE
    e.salario > 50000
),
proyectos_activos AS (
  SELECT
    p.nombre_proyecto,
    p.lider_proyecto
  FROM
    proyectos p
  WHERE
    p.estado = 'Activo'
)
SELECT
  ed.nombre,
  ed.apellido,
  ed.nombre_departamento,
  pa.nombre_proyecto
FROM
  empleados_departamento ed
INNER JOIN
  proyectos_activos pa ON ed.nombre = pa.lider_proyecto;
```

En este caso, se definen dos CTEs: uno para empleados de alto salario y otro para proyectos activos. La consulta principal luego realiza un JOIN entre estos dos CTEs para encontrar empleados que son líderes de proyectos activos. Este método es mucho más fácil de leer y depurar que anidar múltiples subconsultas.

PARTE 3:

Conclusiones.

Como en muchos aspectos de las distintas profesiones, lo mas importante resulta conocer las herramientas disponibles, conocer los casos en que son mejores unas que otras y como contribuyen a obtener los resultados esperados. Sera necesario evaluar el costo-beneficio de utilizar unas u otras en cada caso. Aquí resumimos las características, para identificar la mejor opción según las condiciones de uso.

Diferencias Clave: JOIN vs. UNION

Característica	JOIN	UNION
Propósito	Combinar columnas de diferentes tablas basándose en una condición.	Combinar filas de diferentes conjuntos de resultados.
Dirección	Unión horizontal (más columnas).	Unión vertical (más filas).
Esquema	Las tablas pueden tener estructuras totalmente diferentes .	Todas las SELECT deben tener el mismo número de columnas y tipos de datos compatibles.
Condición	Siempre usa una condición ON para relacionar las tablas.	Nunca usa una condición ON .
Duplicados	Muestra los datos relacionados tal cual, no elimina filas duplicadas de las tablas originales.	UNION sí elimina duplicados del resultado final. UNION ALL los mantiene.
Resultado	Crea una "tabla ancha" con información de varias fuentes.	Crea una "tabla larga" con registros de varias consultas.

Resumen en una Imagen Mental

- **JOIN:** Imagina dos hojas de cálculo (Excel) una al lado de la otra. Unes la información de una fila de la primera hoja con la información correspondiente de la segunda hoja, usando un ID como pegamento.
- **UNION:** Imagina dos listas de nombres en dos hojas diferentes. Quieres juntar todos los nombres en una sola lista larga, poniendo la segunda lista debajo de la primera.

Cómo Diferenciar Cuándo Usar INNER, LEFT o RIGHT JOIN

La elección depende **completamente de la pregunta que quieras hacerle a tus datos**. Aquí tienes una guía infalible:

1. **INNER JOIN:** "Solo lo que coincide"

- **Pregunta clave:** "¿Qué elementos de la Tabla A **tienen una relación confirmada** con la Tabla B?"
- **Usalo cuando:** Solo te interesan los registros que existen en **ambas** tablas. Es el más común y generalmente el más eficiente.

- **Ejemplo práctico:** "Lista todos los **pedidos que han sido pagados**." (Un pedido sin pago no te interesa para este reporte).
2. **LEFT JOIN:** "Todo de A, y lo que coincida de B"
- **Pregunta clave:** "Muéstrame **todos** los elementos de la Tabla A, y si tienen algo en la Tabla B, dame ese dato también. Si no, déjalo en blanco."
- **Usalo cuando:** Tu foco principal es la primera tabla (la de la izquierda), y la segunda es información opcional o complementaria. Es extremadamente útil para encontrar "lo que falta".
- **Ejemplo práctico 1 (info opcional):** "Lista **todos los clientes** y, si tienen un pedido, muestra su fecha." (Un cliente sin pedidos seguirá apareciendo en la lista).
- **Ejemplo práctico 2 (encontrar faltantes):** "¿Qué clientes **NUNCA** han hecho un pedido?" (Usas `WHERE tabla_b.id IS NULL`).
- 3. **RIGHT JOIN:** "Todo de B, y lo que coincida de A"
- **Pregunta clave:** "Muéstrame **todos** los elementos de la Tabla B, y si tienen algo en la Tabla A, dame ese dato también."
- **Usalo cuando:** Es exactamente el caso inverso al LEFT JOIN. **En la práctica, se usa MUCHO MENOS.** ¿Por qué? Porque simplemente puedes cambiar el orden de las tablas y usar un LEFT JOIN. Es más intuitivo pensar siempre en la tabla principal de la que quieres todos los registros y ponerla primero.
Consejo: Casi siempre puedes evitar el RIGHT JOIN reorganizando tus tablas. En lugar de `A RIGHT JOIN B`, escribe `B LEFT JOIN A`. El resultado es el mismo, pero es mucho más fácil de leer y entender.